

Deep Dive — Inference, Decoding & Adaptation

Part 04 — 📖 Large Language Models · 04-llm/notes/02-inference-and-adaptation.md

Everything in this file is a knob you control at runtime or a decision you make with a budget.

1. What generation actually is

At each step the model outputs a vector of **logits** — one real number per vocabulary entry. Then:

```
logits (V,) → [temperature] → [repetition penalties] → [top-k / top-p / min-p filter]
           → softmax → probability distribution → sample → token → append → repeat
```

Understanding this pipeline makes every sampling parameter obvious instead of magical.

2. The decoding parameters, precisely

Temperature T

Divide logits by T before softmax. - $T \rightarrow 0$: greedy/deterministic, the argmax. Repetitive but safe. - $T = 1$: the model's native distribution. - $T > 1$: flattened distribution, more surprising, more incoherent.

Task	Temperature
Extraction, classification, tool-call arguments, SQL	0
Factual Q&A, summarisation	0–0.3
General assistant chat	0.5–0.8
Brainstorming, creative writing	0.8–1.2

Top-k

Keep the k highest-probability tokens, renormalise, sample. Crude: $k=50$ is too many when the model is confident and too few when it's genuinely uncertain.

Top-p (nucleus sampling)

Keep the smallest set of tokens whose cumulative probability $\geq p$. **Adaptive** — the candidate set shrinks when the model is confident. $p = 0.9-0.95$ is a sane default. Generally preferred over top-k.

Min-p

Keep tokens with probability $\geq \text{min_p} \times \text{p_max}$. Behaves better than top-p at high temperature; increasingly the default in open-model serving stacks.

Repetition / frequency / presence penalties

Subtract from the logits of tokens already produced. Useful against degenerate loops; overdone, they push the model away from necessary words (articles, a repeated identifier in code). **Never apply a repetition penalty to structured output or code.**

Do not tune temperature and top-p simultaneously as if they're independent. Pick a temperature for the task, leave top-p at 0.9–1.0, and change one thing at a time. Most "prompt engineering" regressions are actually sampling-parameter regressions.

Structured / constrained decoding

Mask logits so only tokens valid under a grammar or JSON schema can be sampled. This makes malformed JSON *impossible* rather than unlikely. Implementations: Outlines, XGrammar, llama.cpp GBNF, and the "structured output" / "JSON mode" features of the major APIs. **If you parse model output, use this.**

3. The adaptation decision — in cost order

Try these in order. Stop at the first one that works.

Approach	Setup cost	Per-call cost	Best for	Worst for
Prompting	minutes	baseline	almost everything; always start here	strict style, huge private corpora
Few-shot prompting	hours	+tokens	format consistency, narrow tasks	many examples don't fit
RAG	days	+retrieval + tokens	private/current knowledge, citations, auditability	teaching new <i>skills</i> or style
Fine-tuning (LoRA)	days	cheap at scale	style, format, a narrow task, latency/cost reduction	adding facts (it does this badly)
Full fine-tune	weeks	cheap at scale	deep domain adaptation with lots of data	most teams, most of the time
Continued pretraining	months, \$\$\$	—	a genuinely new domain/language	virtually everyone

The rule that saves the most money: *fine-tuning teaches behaviour; retrieval supplies knowledge.* Teams reach for fine-tuning to fix hallucination and it does not work. They should have built RAG.

LoRA, concretely

Freeze the base weights W . Learn a low-rank update $\Delta W = BA$, with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll d$. Train ~ 0.1 – 1% of the parameters. Merge at inference for zero added latency, or hot-swap adapters to serve many fine-tunes from one base model.

- $r = 8$ – 16 for style/format; $r = 32$ – 64 for harder tasks. $\alpha \approx 2r$ is a common starting point.
- Apply to attention projections at minimum; adding the FFN projections usually helps.
- QLoRA = LoRA on a 4-bit quantized base. This is what lets people fine-tune a 70B model on one GPU.

4. Quantization

Store weights (and sometimes activations and KV cache) in fewer bits.

Format	Bits	Typical quality loss	Notes
FP16 / BF16	16	none (baseline)	bf16 is the safe training/serving default
FP8	8	very small	native on recent datacentre GPUs
INT8	8	small	widely supported
4-bit (GPTQ, AWQ, NF4)	4	small→moderate, task-dependent	the practical sweet spot for local models
2–3 bit	2–3	significant	research / extreme memory constraints

Rules of thumb: memory for weights $\approx \text{params} \times \text{bytes_per_param}$. A 70B model is ~ 140 GB at bf16, ~ 35 GB at 4-bit. **A larger model quantized to 4-bit usually beats a smaller model at bf16** at the same memory budget. Always re-run your evals after quantizing — losses are task-dependent, and reasoning and code degrade first.

5. Serving performance — the four numbers that matter

- **TTFT** (time to first token) — dominated by *prefill*: compute-bound, scales with prompt length.
- **TPOT / ITL** (time per output token) — dominated by *decode*: memory-bandwidth-bound, sequential.
- **Throughput** (tokens/sec across all requests) — what your cost per token depends on.
- **Concurrency** — capped by KV cache memory (Module 03 §8 [03-nlp/notes/02-transformers.md]).

Techniques that move them: - **Continuous batching** — the single biggest throughput win in real serving (vLLM, TGI, TensorRT-LLM). - **PagedAttention** — KV cache in non-contiguous pages; kills fragmentation, raises concurrency. - **Prefix / prompt caching** — reuse the KV cache for a shared prompt prefix. Huge for system prompts, few-shot blocks and long documents. Offered by the major APIs; **structure your prompts so the stable part comes first**, or you get none of the benefit. - **Speculative decoding** — a small draft model proposes k tokens; the big model verifies them in one forward pass. 2 – $3\times$ faster decode with **provably identical output distribution**. Variants: Medusa, EAGLE, n-gram/prompt lookup. - **Distillation** — train a

small student on a large teacher's outputs. The most reliable way to cut inference cost by an order of magnitude once you have a working expensive pipeline.

6. Cost modelling (do this before you build)

```
cost_per_request = (input_tokens * input_price_per_token)
                  + (output_tokens * output_price_per_token)
```

Facts that dominate real bills: - **Output tokens are typically 3–5× the price of input tokens.** Verbosity is expensive; ask for terse output. - **Cached input tokens are dramatically cheaper.** Put the stable prefix first and keep it byte-identical. - RAG multiplies input tokens. Retrieving 20 chunks of 500 tokens adds 10k input tokens *per request*. - Agent loops multiply everything: 10 turns of a growing transcript is quadratic-ish in cost. - Reasoning models bill for thinking tokens you never see.

Run `../code/llm_cost_calculator.py` with your own numbers before committing to an architecture. The cheapest optimisation is almost always "use a smaller model for the easy 80%".

7. Evaluating an LLM system (preview of Module 15)

Do not ship on vibes. The minimum viable eval: 1. **20–50 real examples** with expected behaviour, hand-written by someone who knows the domain. 2. **Deterministic checks first** — schema validity, required fields, forbidden strings, latency, cost. 3. **LLM-as-judge with a rubric** for the subjective parts, with the judge's agreement against humans measured. 4. **Run on every prompt change.** A prompt is code; it deserves regression tests. 5. **Log production traffic** and mine it for new eval cases. Your eval set should grow weekly.

8. Practical checklist before you ship

- [] Temperature and sampling set deliberately per task, not copied from a tutorial.
- [] Structured outputs enforced by schema, not hope.
- [] Prompt versioned in source control with the eval results that justified it.
- [] Stable prefix first, for prompt caching.
- [] Timeouts, retries with jitter, and a fallback model configured.
- [] Token budget per request enforced in code, plus a hard spend alert.
- [] Every model call logged with prompt hash, model version, tokens, latency and cost (Module 15 [15-ai-evals]).
- [] Untrusted content clearly delimited and treated as data (Module 13 [13-ai-security]).